
WEB ARCHITECTURE LAB 01

Model-View-Controller (MVC) Architecture

Building a Library Book Manager

Prepared by:
Njayou Youssouf
Lecturer, IUT CSE



Department of Computer Science and Engineering
Islamic University of Technology
May 7th, 202

1. Lab Objectives

By the end of this lab, you will be able to:

- **Understand** the Model-View-Controller (MVC) architectural pattern and the role of each component in separating concerns.
- **Identify** which responsibilities belong to the Model, View, and Controller in a given application.
- **Implement** a multi-class Java console application that follows the MVC pattern.
- **Trace** the flow of data and control through an MVC application and explain each step.
- **Extend** an existing MVC application with new features while preserving separation of concerns.

2. Prerequisites

- **Java SE 11 or later** installed (JDK, not just JRE).
- A Java IDE or text editor (IntelliJ IDEA, Eclipse, VS Code with Java Extension Pack, or similar).
- Basic Java programming: classes, constructors, ArrayList, for-each loops, formatted output (printf).
- Familiarity with concepts from **Lecture 01** (Web Architecture, Tiered Architecture, and MVC overview).

3. Background: The MVC Pattern

3.1 What Is MVC?

The Model-View-Controller (MVC) pattern divides an application into three interconnected components, each with a single, clear responsibility. This separation makes code easier to develop in teams, test independently, and maintain over time.

3.2 The Three Components

Component	Responsibility	Examples
Model	Stores and manages data. Contains business rules and logic (e.g., grade calculations, validation). Has no knowledge of how data is displayed or how the user interacts.	Database access classes, data objects (Student, Order), calculation engines, API data layers.
View	Handles all user-facing interaction: displaying information and collecting input. Contains no business logic and does not directly access data storage.	Console output/input, GUI forms, HTML/JSP pages, React components, mobile screens.

Controller	Acts as the intermediary. Receives user actions from the View, calls the Model to get or update data, and decides what the View should display next.	Servlets, Spring Controllers, request handlers, event listeners, menu handlers.
-------------------	--	---

3.3 The MVC Flow

The interaction follows a repeating cycle:

1. **User interacts with the View** – types input, selects a menu option, clicks a button.
2. **View forwards the action to the Controller** – the View itself does not decide what happens next.
3. **Controller processes the request** – it calls one or more methods on the Model.
4. **Model performs the operation** – retrieves, computes, or updates data and returns results.
5. **Controller passes results to the View** – it decides what information the View should show.
6. **View displays the updated information** – the cycle repeats from Step 1.

Key rule: the View and the Model should never communicate directly. All communication passes through the Controller.

3.4 Why Use MVC?

Advantages	Disadvantages
Separation of Concerns – each component has one job, making code cleaner.	Added complexity for very small programs where a single file would suffice.
Parallel development – one developer works on the View (UI), another on the Model (data).	Requires the team to understand the pattern before they can contribute effectively.
Testability – each component can be unit-tested in isolation.	More files and classes to manage as the application grows.
Reusability – swap the View (console → web) without changing the Model or Controller logic.	Debugging may involve tracing through multiple layers.

4. Provided Source Code: Library Book Manager

The application below manages a small library catalogue. It allows the user to view all books, search by ISBN, check out a book, and see how many books are currently available. The code is split into five files following MVC. Study each file and pay attention to the comments.

4.1 Book.java (Model – Data Class)

This is a plain data class (sometimes called a POJO – Plain Old Java Object). It holds the data for one book and includes the availability status as a business rule.

```
/**
 * Book.java - A simple data class representing one book.
 * This is part of the MODEL layer.
 */
public class Book {

    // — Fields —
    private String isbn;           // e.g. "978-0134685991"
    private String title;         // e.g. "Effective Java"
    private String author;        // e.g. "Joshua Bloch"
    private boolean checkedOut;    // true if currently borrowed

    // — Constructor —
    public Book(String isbn, String title,
                String author) {
        this.isbn      = isbn;
        this.title      = title;
        this.author     = author;
        this.checkedOut = false; // new books start available
    }

    // — Getters —
    public String getIsbn()      { return isbn; }
    public String getTitle()     { return title; }
    public String getAuthor()    { return author; }
    public boolean isCheckedOut() { return checkedOut; }

    // — Checkout / Return —
    public void checkOut() { this.checkedOut = true; }
    public void returnBook() { this.checkedOut = false; }

    /**
     * Returns a human-readable availability status.
     * This logic belongs in the Model because it is
     * a business rule, not a display concern.
     */
    public String getStatus() {
        return checkedOut ? "Checked Out" : "Available";
    }
}
```

4.2 Library.java (Model – Data Manager)

This is the main Model class. It manages the collection of Book objects and provides methods to query and modify the data. Notice that it has **no `System.out.println` calls** – it does not know or care about the user interface.

```
import java.util.ArrayList;
import java.util.List;

/**
 * Library.java - Manages a collection of Books.
 * This is the main MODEL class. It stores data and
 * provides methods to query and modify it.
 * It has NO knowledge of how data is displayed.
 */
public class Library {

    // The list that holds all book records
    private final List<Book> books = new ArrayList<>();

    // — Constructor: pre-load some sample data —
    public Library() {
        books.add(new Book("978-0134685991",
            "Effective Java", "Joshua Bloch"));
        books.add(new Book("978-0596009205",
            "Head First Design Patterns",
            "Eric Freeman"));
        books.add(new Book("978-0132350884",
            "Clean Code", "Robert C. Martin"));
        books.add(new Book("978-0201633610",
            "Design Patterns", "Gang of Four"));
    }

    /** Returns a copy of the full book list. */
    public List<Book> getAllBooks() {
        return new ArrayList<>(books);
    }

    /**
     * Searches for a book by ISBN.
     * Returns null if not found.
     */
    public Book findByIsbn(String isbn) {
        for (Book b : books) {
            if (b.getIsbn().equals(isbn)) {
                return b;
            }
        }
        return null; // not found
    }

    /** Adds a new book to the library. */
    public void addBook(Book book) {
        books.add(book);
    }
}
```

```
/**
 * Counts the number of available books.
 * Returns 0 if no books exist.
 */
public int getAvailableCount() {
    int count = 0;
    for (Book b : books) {
        if (!b.isCheckedOut()) {
            count++;
        }
    }
    return count;
}

/** Returns the total number of books. */
public int getTotalCount() {
    return books.size();
}
}
```

4.3 LibraryView.java (The View)

This class handles **all user interaction**: printing menus, displaying formatted tables, showing messages, and collecting input. It never accesses the Library directly – it receives data as method parameters.

```
import java.util.List;
import java.util.Scanner;

/**
 * LibraryView.java - Handles ALL user interaction.
 * This is the VIEW layer. It displays information
 * and collects input. It contains NO business logic
 * and does NOT access data directly.
 */
public class LibraryView {

    private final Scanner scanner = new Scanner(System.in);

    // =====
    // OUTPUT METHODS (display to user)
    // =====

    /** Displays the main menu and returns the choice. */
    public int showMenu() {
        System.out.println();
        System.out.println("=====");
        System.out.println("    LIBRARY BOOK MANAGER");
        System.out.println("=====");
        System.out.println("1. View all books");
        System.out.println("2. Search book by ISBN");
        System.out.println("3. Check out a book");
        System.out.println("4. View availability summary");
        System.out.println("5. Exit");
        System.out.print("Enter your choice: ");
        return readInt();
    }

    /** Displays a list of books in a formatted table. */
    public void showBookList(List<Book> books) {
        System.out.println();
        System.out.printf("%-18s %-28s %-20s %s\n",
            "ISBN", "Title", "Author", "Status");
        System.out.println("-".repeat(78));
        for (Book b : books) {
            System.out.printf("%-18s %-28s %-20s %s\n",
                b.getIsbn(), b.getTitle(),
                b.getAuthor(), b.getStatus());
        }
    }

    /** Displays details for a single book. */
    public void showBookDetails(Book book) {
        System.out.println();
        System.out.println("--- Book Found ---");
        System.out.println("ISBN:    " + book.getIsbn());
    }
}
```

```
        System.out.println("Title: " + book.getTitle());
        System.out.println("Author: " + book.getAuthor());
        System.out.println("Status: " + book.getStatus());
    }

    /** Displays the availability summary. */
    public void showAvailability(int available,
                               int total) {
        System.out.printf(
            "%nAvailability: %d of %d books available%n",
            available, total);
    }

    /** Displays an error or informational message. */
    public void showMessage(String message) {
        System.out.println(message);
    }

    // =====
    // INPUT METHODS (collect from user)
    // =====

    /** Prompts for and returns a book ISBN. */
    public String askIsbn() {
        System.out.print("Enter book ISBN: ");
        return scanner.nextLine().trim();
    }

    /** Safely reads an integer from the user. */
    private int readInt() {
        while (!scanner.hasNextInt()) {
            scanner.next(); // discard bad input
            System.out.print("Invalid. Enter a number: ");
        }
        int value = scanner.nextInt();
        scanner.nextLine(); // consume newline
        return value;
    }
}
```


4.4 LibraryController.java (The Controller)

The Controller is the **orchestrator**. It holds references to both the Model and the View. It runs the main loop, interprets user choices, calls the Model for data, and tells the View what to display. Notice it **never prints directly** and **never stores data**.

```
/**
 * LibraryController.java - The CONTROLLER layer.
 * It connects the View and the Model:
 *   - Receives user actions from the View
 *   - Calls the Model to get or change data
 *   - Tells the View what to display
 *
 * The Controller NEVER prints to the console directly.
 * The Controller NEVER stores data itself.
 */
public class LibraryController {

    // References to Model and View
    private final Library model;
    private final LibraryView view;

    /** Constructor: receives both dependencies. */
    public LibraryController(Library model,
                            LibraryView view) {
        this.model = model;
        this.view = view;
    }

    /**
     * Main application loop.
     * Keeps showing the menu until the user exits.
     */
    public void run() {
        boolean running = true;
        while (running) {
            // Step 1: View shows menu, returns choice
            int choice = view.showMenu();

            // Step 2: Controller decides what to do
            switch (choice) {
                case 1:
                    handleViewAll();
                    break;
                case 2:
                    handleSearchByIsbn();
                    break;
                case 3:
                    handleCheckOut();
                    break;
                case 4:
                    handleAvailability();
                    break;
                case 5:
                    running = false;
                    view.showMessage("Goodbye!");
            }
        }
    }

    private void handleViewAll() {
        // ...
    }

    private void handleSearchByIsbn() {
        // ...
    }

    private void handleCheckOut() {
        // ...
    }

    private void handleAvailability() {
        // ...
    }
}
```

```

        break;
    default:
        view.showMessage(
            "Invalid choice. Try again.");
    }
}

// — Private handler methods —

/** Gets all books from Model, sends to View. */
private void handleViewAll() {
    view.showBookList(model.getAllBooks());
}

/** Asks View for ISBN, queries Model, shows result. */
private void handleSearchByIsbn() {
    String isbn = view.askIsbn();
    Book book = model.findByIsbn(isbn);
    if (book != null) {
        view.showBookDetails(book);
    } else {
        view.showMessage(
            "No book found with ISBN: " + isbn);
    }
}

/** Asks for ISBN, checks out if available. */
private void handleCheckOut() {
    String isbn = view.askIsbn();
    Book book = model.findByIsbn(isbn);
    if (book == null) {
        view.showMessage(
            "No book found with ISBN: " + isbn);
    } else if (book.isCheckedOut()) {
        view.showMessage(
            "Sorry, that book is already checked out.");
    } else {
        book.checkOut();
        view.showMessage("Successfully checked out: "
            + book.getTitle());
    }
}

/** Gets counts from Model, sends to View. */
private void handleAvailability() {
    int available = model.getAvailableCount();
    int total = model.getTotalCount();
    view.showAvailability(available, total);
}
}

```

4.5 Main.java (Entry Point)

This class performs **dependency injection**: it creates the three components and wires them together. It contains no business logic.

```
/**
 * Main.java - Entry point of the application.
 *
 * This class wires the MVC components together:
 * 1. Creates the Model (data layer)
 * 2. Creates the View (presentation layer)
 * 3. Creates the Controller (logic layer),
 *    passing it references to Model and View
 * 4. Starts the Controller
 *
 * Notice: Main does NOT contain any business logic.
 * It only performs dependency injection.
 */
public class Main {

    public static void main(String[] args) {
        // 1. Create the Model (data + business rules)
        Library model = new Library();

        // 2. Create the View (user interface)
        LibraryView view = new LibraryView();

        // 3. Create the Controller, inject dependencies
        LibraryController controller =
            new LibraryController(model, view);

        // 4. Start the application loop
        controller.run();
    }
}
```

5. Expected Output

When the program runs, you should see output similar to the following:

```
=====
LIBRARY BOOK MANAGER
=====
1. View all books
2. Search book by ISBN
3. Check out a book
4. View availability summary
5. Exit
Enter your choice: 1

ISBN          Title          Author          Status
-----
978-0134685991 Effective Java  Joshua Bloch    Available
978-0596009205 Head First Design Patterns Eric Freeman    Available
978-0132350884 Clean Code     Robert C. Martin Available
978-0201633610 Design Patterns Gang of Four     Available

=====
LIBRARY BOOK MANAGER
=====
1. View all books
2. Search book by ISBN
3. Check out a book
4. View availability summary
5. Exit
Enter your choice: 3
Enter book ISBN: 978-0132350884
Successfully checked out: Clean Code

=====
LIBRARY BOOK MANAGER
=====
1. View all books
2. Search book by ISBN
3. Check out a book
4. View availability summary
5. Exit
Enter your choice: 4

Availability: 3 of 4 books available
```

6. Lab Procedure

Part A: Setup and Run

- Create a new Java project in your IDE.
- Create the five Java files from Section 4: **Book.java**, **Library.java**, **LibraryView.java**, **LibraryController.java**, and **Main.java**.
- Compile and run the program. Verify that:
 - Option 1 displays all four books with correct statuses (all "Available" initially).
 - Option 2 with ISBN **978-0132350884** shows "Clean Code" by Robert C. Martin.
 - Option 2 with a non-existent ISBN (e.g., **000-0000000000**) shows an error message.
 - Option 3 checks out a book and changes its status to "Checked Out".
 - Option 4 displays the correct availability summary.
 - Option 5 exits the program with a goodbye message.

Part B: Understanding the MVC Flow

Answer the following questions by studying the source code.

Q1: When the user selects option 3 (Check out a book) and enters a valid ISBN, trace the exact sequence of method calls starting from `LibraryController.run()`. List each method call in order and state which component (Model, View, or Controller) it belongs to.

Q2: In `LibraryView.showBookList()`, the method receives a `List<Book>` parameter. Why does the View receive data as a parameter instead of calling `model.getAllBooks()` directly? What MVC principle does this enforce?

Q3: The `getStatus()` method is defined in `Book.java` (the Model). Could this method be placed in `LibraryView.java` instead? Explain why or why not, considering MVC responsibilities.

Q4: If you wanted to replace the console interface with a graphical (GUI) interface, which classes would you need to change? Which classes would remain completely untouched? Explain why.

Q5: Draw a sequence diagram (on paper or using a diagramming tool) showing the complete interaction between Main, Controller, View, and Model when the user selects option 3 and enters the ISBN "978-0132350884" for a book that is currently available.

Part C: Extending the Application

Implement the following features. For each task, think carefully about which component (Model, View, or Controller) each change belongs to.

Task 1: Add More Books

Add at least three more books to the Library constructor with different authors (include at least one book that starts as already checked out by calling `checkOut()` after creation). Run the program and verify they appear in the list with correct statuses.

Task 2: Return a Book

Add a new menu option (option 6: "Return a book") that allows the user to return a previously checked-out book. This requires changes to all three layers:

- **View:** Add a method to ask for the ISBN of the book being returned.
- **Model:** The Book class already has `returnBook()`. No Model changes needed.
- **Controller:** Add a `handleReturn()` method that asks the View for the ISBN, finds the book in the Model, calls `returnBook()` if it is currently checked out, and confirms via the View. Handle the case where the book is already available.

Task 3: Add a New Book

Add menu option 7: "Add new book". The View should collect the ISBN, title, and author from the user. The Controller should pass this data to the Model. Verify the new book appears in the list after being added.

Task 4: Search by Author

Add menu option 8: "Search by author". The View asks the user to enter an author name (or part of it). Add a method `getBooksByAuthor(String author)` to Library that returns all books whose author field contains the search string (case-insensitive). The View displays the matching books using the existing `showBookList()` method.

Task 5: Display Checked-Out Books Only

Add menu option 9: "View checked-out books". Add a method `getCheckedOutBooks()` to Library that returns only books with a status of "Checked Out". If no books are currently checked out, show an appropriate message through the View.